

SQL

Bases de données relationnelles

Traineau Julien

o. Introduction — À quoi sert SQL ?

Tous les sites web modernes ont besoin de **stocker des données** : des utilisateurs, des messages, des photos, des documents, des scores de jeux... Sans base de données, tout serait perdu à chaque redémarrage du serveur.

Prenons un exemple concret : le site que j'ai développé, **Chat React**. C'est une plateforme web où les élèves peuvent :

-  Discuter en temps réel dans des salons ou en privé
-  Jouer à UNO et aux Échecs multijoueur
-  Partager des documents
-  Se faire modérer (mute, ban) par un administrateur

Chacune de ces fonctionnalités a besoin d'une base de données SQL. Sans SQL, il serait impossible de :

- Retrouver ton compte quand tu te connectes
- Afficher les anciens messages du chat

- Sauvegarder ton score de victoire au UNO
- Empêcher un utilisateur banni de revenir

LE SAVIEZ-VOUS ?

SQL (Structured Query Language) est utilisé par **presque tous les sites web au monde** : Facebook, Instagram, YouTube, Amazon, Netflix... Tous stockent leurs données dans des bases de données relationnelles et les interrogent avec SQL.

1. Bases de données : généralités

Une **base de données** est un ensemble organisé de données structurées, stockées de manière permanente et accessibles simultanément par plusieurs programmes ou utilisateurs.

Il existe plusieurs types de bases de données (hiérarchique, réseau), mais la plus utilisée aujourd'hui est la base de données **relationnelle**. Elle organise les données en **tables** reliées entre elles.

Le SGBD — Système de Gestion de Base de Données

Pour manipuler une base, on utilise un **SGBD**. C'est le logiciel qui permet de créer, organiser, interroger et mettre à jour les données.

SGBD populaires

 MySQL

 PostgreSQL

 Oracle

 SQLite

Dans mon projet

PostgreSQL (hébergé sur Neon)

Sequelize (ORM Node.js)

Le langage SQL

L'accès aux données se fait grâce à des **requêtes SQL**. Une requête SQL est une instruction qui permet de :

- **Lire** des données (SELECT)

- **Insérer** des données (INSERT)
- **Modifier** des données (UPDATE)
- **Supprimer** des données (DELETE)

exemple_concret.sql

```
-- Sur mon site, quand un utilisateur se connecte :  
-- Le serveur fait cette requête SQL :  
SELECT * FROM  
"Users" WHERE email =  
'julientraineau17@gmail.com';  
  
-- Résultat : la ligne de l'utilisateur julien  
-- Le serveur peut alors comparer le mot de passe  
-- et autoriser (ou refuser) la connexion.
```

2. Le modèle relationnel

Le **modèle relationnel** organise les données sous forme de **tables** (aussi appelées relations). Les tables sont reliées entre elles par des **clés**.

Vocabulaire essentiel

TERME	DÉFINITION
Table	Ensemble de données sur un même sujet (ex : les utilisateurs)
Attribut	Une colonne (ex : username, email)
N-uplet (ligne)	Un enregistrement complet (ex : un utilisateur)
Domaine	Type de valeurs possibles (entier, texte, date...)
Clé primaire (PK)	Attribut unique qui identifie chaque ligne
Clé étrangère (FK)	Référence vers la clé primaire d'une autre table

Exemple : la table "Users" de mon site

Sur mon site, chaque utilisateur est enregistré dans une table `Users`. Voici sa structure :

Users	
	<code>id</code>
	<code>username</code>
	<code>email</code>
	<code>password</code>
	<code>role</code>
	<code>createdAt</code>

→ La clé primaire `id` est un entier auto-incrémenté (1, 2, 3...). Elle identifie chaque utilisateur de manière unique.

3. Comment créer une table en SQL

Pour créer une table en SQL, on utilise la commande `CREATE TABLE`. Chaque attribut a un **type** (texte, entier, date...) et des **contraintes** (obligatoire, unique...).

Les types de données principaux

TYPE	UTILISATION	EXEMPLE
INTEGER / SERIAL	Nombre entier	42, 2026
VARCHAR(n)	Texte de longueur limitée (n caractères)	'Dupont '
TEXT	Texte de longueur illimitée	Messages de chat
BOOLEAN	Vrai ou faux	TRUE, FALSE
TIMESTAMP	Date et heure	'2026-09-30 14:30:00'

Les contraintes principales

CONTRAINTE	SIGNIFICATION
PRIMARY KEY	Identifiant unique de la ligne
NOT NULL	Valeur obligatoire (ne peut pas être vide)
UNIQUE	La valeur doit être unique dans la colonne
DEFAULT v	Valeur par défaut si rien n'est précisé
REFERENCES t(c)	Clé étrangère vers la table t, colonne c

Exemple 1 : créer la table Users

Voici la vraie commande SQL utilisée sur mon site pour créer la table des utilisateurs :

create_users.sql

```
CREATE TABLE "Users" (  
  id SERIAL PRIMARY KEY,  
  username VARCHAR(255)  
  NOT NULL UNIQUE, email  
  VARCHAR(255)  
  NOT NULL UNIQUE, password  
  VARCHAR(255)  
  NOT NULL, role  
  VARCHAR(20)  
  NOT NULL DEFAULT  
  'membre', online  
  BOOLEAN NOT NULL DEFAULT  
  FALSE, banned BOOLEAN  
  NOT NULL DEFAULT FALSE,  
  "unoWins" INTEGER  
  NOT NULL DEFAULT 0,  
  "chessWins" INTEGER  
  NOT NULL DEFAULT 0,  
  "createdAt" TIMESTAMP  
  NOT NULL DEFAULT NOW() );
```

→ *SERIAL* en PostgreSQL est un entier **auto-incrémenté** : chaque nouvel utilisateur reçoit automatiquement un nouvel ID (1, 2, 3...).

Exemple 2 : créer la table Messages avec une clé étrangère

La table Messages a besoin de savoir **qui** a écrit le message. Elle utilise donc une **clé étrangère** vers la table Users.

```
create_messages.sql
```

```
CREATE TABLE
"Messages" ( id SERIAL
PRIMARY KEY, content
TEXT NOT NULL,
-- Clé étrangère : identifie l'auteur du message
"senderId" INTEGER
NOT NULL REFERENCES
"Users"(id)
ON DELETE CASCADE,
-- Clé étrangère : identifie le salon (null = message privé)
"roomId" INTEGER
REFERENCES "Rooms"(id)
ON DELETE CASCADE, "createdAt"
TIMESTAMP
NOT NULL DEFAULT NOW() );
```

COMPRENDRE REFERENCES

REFERENCES "Users"(id) veut dire : *"la valeur de cette colonne doit exister dans la colonne id de la table Users"*.

ON DELETE CASCADE : si un utilisateur est supprimé, tous ses messages le seront aussi automatiquement.

4. Le schéma relationnel

Le **schéma relationnel** est la représentation formelle d'une base. On note la clé primaire soulignée et les clés étrangères suivies d'un #.

Schéma complet de mon site Chat React

Users (id, username, email, password, role, online, banned, unoWins, chessWins)

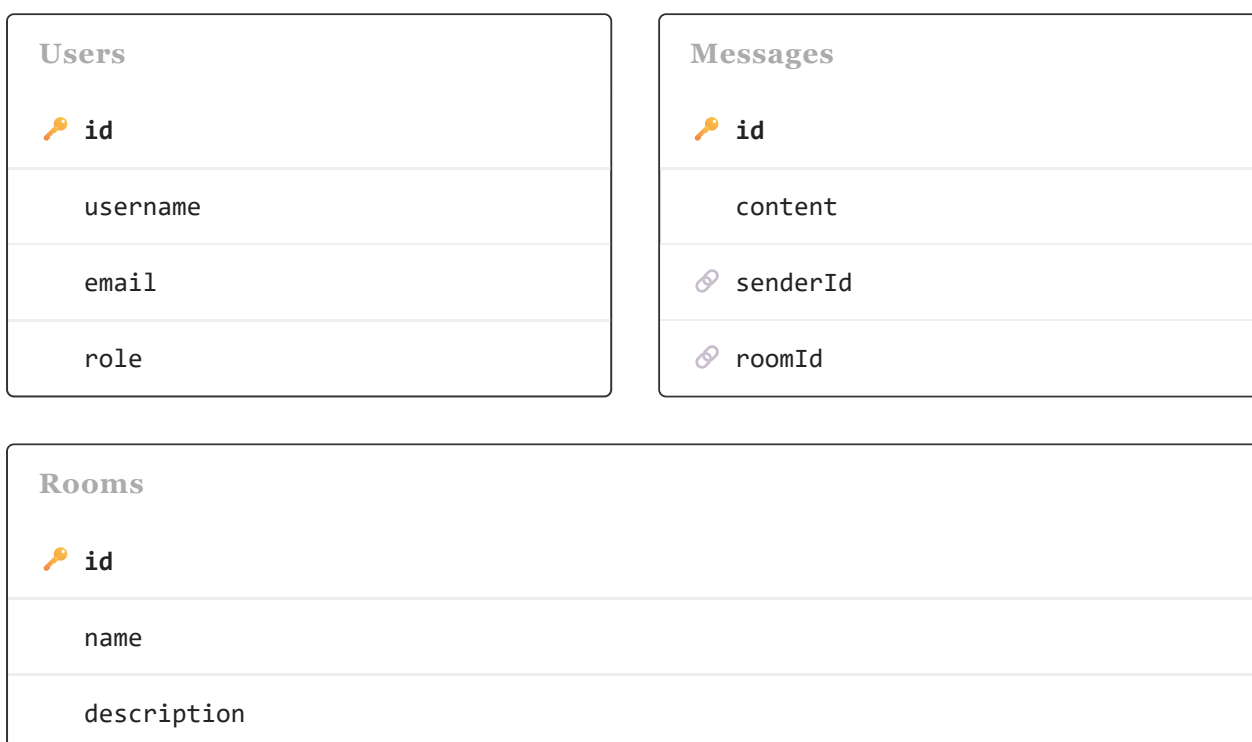
Rooms (id, name, description, createdById#)

Messages (id, content, type, senderId#, receiverId#, roomId#)

Documents (id, name, url, size, uploadedById#)

BanList (id, email, reason)

Vue visuelle des relations



→ La table Messages fait le lien entre Users et Rooms grâce à ses deux clés étrangères. C'est ce qui permet de retrouver "qui a écrit quoi dans quel salon".

5. Insérer des données — INSERT

Une fois les tables créées, on peut y **ajouter des données** avec la commande INSERT INTO.

Exemple : créer un compte utilisateur

Sur mon site, quand quelqu'un s'inscrit, voici la requête SQL qui est exécutée :

```
insert_user.sql
```

```
INSERT INTO "Users"
(username, email, password, role)
VALUES
('julien',
'julientraineau17@gmail.com',
'$2a$10$HHvaDQSTQGZqvX...',
'admin');

-- Le mot de passe est haché par bcrypt avant d'être stocké
-- C'est pour ça qu'il ressemble à une longue chaîne étrange
```

REQUÊTE INSERT AVEC AUTO-INCRÉMENTATION

On ne précise **pas** l'id dans l'INSERT : PostgreSQL le génère automatiquement grâce à SERIAL.

Exemple : envoyer un message dans le chat

```
insert_message.sql
```

```
-- Utilisateur 1 envoie "Salut !" dans le salon général (id 1)
INSERT INTO "Messages"
(content, "senderId", "roomId")
VALUES
('Salut !', 1,
1);
```

6. Lire des données — SELECT

La commande SELECT est la plus utilisée en SQL : elle permet de **lire** des données.

6.1 — Projection : choisir les colonnes

```
select_projection.sql

-- Récupérer uniquement pseudo et email de tous les
  utilisateurs
SELECT username, email
FROM "Users";

-- Récupérer TOUTES les colonnes
SELECT * FROM
"Users";

-- Retirer les doublons (par exemple les rôles existants)
SELECT DISTINCT role
FROM "Users";
```

6.2 — Sélection : filtrer les lignes

```
select_where.sql

-- Trouver un utilisateur par son email (connexion)
SELECT * FROM
"Users" WHERE email =
'julientraineau17@gmail.com';

-- Tous les administrateurs du site
SELECT username, email
FROM "Users"
WHERE role = 'admin';

-- Tous les utilisateurs bannis
SELECT * FROM
"Users" WHERE banned =
TRUE;

-- Utilisateurs ayant au moins 5 victoires UNO
SELECT username, "unoWins"
FROM "Users"
WHERE "unoWins" >=
5;
```

6.3 — Jointure : combiner plusieurs tables

Pour afficher un message **avec le nom de son auteur**, il faut **joindre** la table Messages et la table Users.

select_join.sql

```
-- Récupérer chaque message AVEC Le pseudo de son auteur
```

```
SELECT
```

```
"Messages".content,
```

```
"Users".username,
```

```
"Messages"."createdAt"
```

```
FROM "Messages",
```

```
"Users" WHERE
```

```
"Messages"."senderId" =
```

```
"Users".id AND
```

```
"Messages"."roomId" =
```

```
1 ORDER BY
```

```
"Messages"."createdAt"
```

```
ASC;
```

Comment ça marche ?

DÉCOMPOSITION DE LA JOINTURE

1. On liste les tables dans FROM : Messages, Users
2. On indique la condition de liaison : Messages.senderId = Users.id
3. On ajoute les filtres supplémentaires avec AND (ici, on ne veut que les messages du salon 1)
4. On trie les résultats par date croissante

6.4 — Jointure à 3 tables

Pour afficher les messages avec leur auteur **et** le nom du salon, il faut joindre 3 tables :

```
select_join3.sql
```

```
SELECT
  "Messages".content,
  "Users".username,
  "Rooms".name
AS room_name FROM
  "Messages", "Users",
  "Rooms" WHERE
  "Messages"."senderId" =
  "Users".id AND
  "Messages"."roomId" =
  "Rooms".id;
```

7. Modifier et supprimer

7.1 — Modifier : UPDATE

```
update.sql
```

```
-- Promouvoir un utilisateur en administrateur
UPDATE "Users"
SET role = 'admin'
WHERE id = 8;

-- Bannir un utilisateur
UPDATE "Users"
SET banned = TRUE,
  "bannedReason" = 'Comportement toxique'
WHERE username =
  'troll123';

-- Incrémenter les victoires UNO d'un joueur
UPDATE "Users"
SET "unoWins" = "unoWins" +
  1 WHERE id =
  1;
```

⚠ ATTENTION

Sans WHERE, la commande UPDATE modifierait **TOUTES les lignes** de la table !

7.2 — Supprimer : DELETE

```
delete.sql

-- Supprimer les messages vieux de plus de 7 jours
DELETE FROM "Messages"
WHERE "createdAt" <
NOW() - INTERVAL
'7 days';

-- Supprimer un utilisateur précis
DELETE FROM "Users"
WHERE id = 42;
```

8. Opérateurs et fonctions

Opérateurs de sélection

OPÉRATEUR	SIGNIFICATION	EXEMPLE (SUR MON SITE)
=, <, >	Comparaisons	WHERE "unoWins" > 10
AND, OR, NOT	Logique	WHERE role='admin' AND online=TRUE
IN (...)	Parmi une liste	WHERE role IN ('admin', 'membre')
BETWEEN a AND b	Intervalle	WHERE "unoWins" BETWEEN 5 AND 20
LIKE 'motif'	Motif	WHERE username LIKE 'jul%'
IS NULL	Valeur vide	WHERE "roomId" IS NULL (MP)

Fonctions d'agrégation

FONCTION	RÔLE	EXEMPLE CONCRET
COUNT()	Compter	Nombre total d'utilisateurs inscrits
SUM()	Somme	Total des victoires UNO de tous les joueurs
AVG()	Moyenne	Moyenne des victoires par joueur
MAX()	Maximum	Meilleur joueur d'Échecs

```
stats.sql
```

```
-- Compter Le nombre d'utilisateurs inscrits
```

```
SELECT COUNT(*) FROM  
"Users";
```

```
-- Nombre d'utilisateurs actuellement en ligne
```

```
SELECT COUNT(*) FROM  
"Users" WHERE online =  
TRUE;
```

```
-- Top 10 des joueurs UNO (classement)
```

```
SELECT username, "unoWins"  
FROM "Users"  
ORDER BY "unoWins" DESC  
LIMIT 10;
```

```
-- Meilleur joueur d'Échecs
```

```
SELECT username,  
MAX("chessWins") AS wins  
FROM "Users"  
GROUP BY username  
ORDER BY wins DESC  
LIMIT 1;
```

9. Exercices corrigés

EXERCICE 1 — SUR LA BASE DE MON SITE

Users (id, username, email, role, online, banned, unoWins, chessWins)

Messages (id, content, senderId#, roomId#)

Rooms (id, name, description)

1. Lister tous les utilisateurs triés par pseudo alphabétique.
2. Trouver tous les administrateurs du site.
3. Afficher le nombre de messages envoyés par l'utilisateur numéro 1.
4. Afficher les messages du salon "general" avec le pseudo de leur auteur.
5. Lister les 5 meilleurs joueurs d'Échecs.
6. Compter le nombre d'utilisateurs bannis.

correction.sql

```
-- 1.
SELECT username, email
FROM "Users"
ORDER BY username ASC;

-- 2.
SELECT username, email
FROM "Users"
WHERE role = 'admin';

-- 3.
SELECT COUNT(*) FROM
"Messages"
WHERE "senderId" = 1;

-- 4.
SELECT
"Messages".content,
"Users".username FROM
"Messages", "Users",
"Rooms" WHERE
"Messages"."senderId" =
"Users".id AND
"Messages"."roomId" =
"Rooms".id AND
"Rooms".name =
'general';

-- 5.
SELECT username, "chessWins"
FROM "Users"
ORDER BY "chessWins"
DESC LIMIT
5;

-- 6.
SELECT COUNT(*) FROM
"Users" WHERE banned
= TRUE;
```

EXERCICE 2 — BASE DE BIBLIOTHÈQUE (TYPE BAC)

Auteurs (idAuteur, prenom, nom, dateNaissance)

Livres (idLivre, titre, annee, idAuteur#)

Emprunts (idEmprunt, date, idLivre#)

1. Afficher tous les livres triés par année décroissante.
2. Afficher les livres écrits par Victor Hugo.
3. Compter le nombre total de livres.
4. Afficher les livres avec le nom de leur auteur.

correction.sql

```
-- 1.
SELECT * FROM
"Livres"
ORDER BY annee DESC;

-- 2.
SELECT "Livres".titre
FROM "Livres",
"Auteurs" WHERE
"Livres"."idAuteur" =
"Auteurs"."idAuteur"
AND "Auteurs".nom =
'Hugo' AND
"Auteurs".prenom =
'Victor';

-- 3.
SELECT COUNT(*) FROM
"Livres";

-- 4.
SELECT
"Livres".titre,
"Auteurs".prenom,
"Auteurs".nom FROM
"Livres", "Auteurs"
WHERE
"Livres"."idAuteur" =
"Auteurs"."idAuteur";
```

10. Récapitulatif

COMMANDE SQL	RÔLE	EXEMPLE
CREATE TABLE	Créer une table	CREATE TABLE Users (...)
INSERT INTO	Ajouter une ligne	INSERT INTO Users VALUES (...)
SELECT ... FROM	Lire des données (projection)	SELECT username FROM Users
WHERE	Filtrer (sélection)	WHERE role = 'admin'
FROM t1, t2 WHERE	Joindre plusieurs tables	WHERE t1.id = t2.fk
ORDER BY	Trier	ORDER BY username ASC
UPDATE ... SET	Modifier une ligne	UPDATE Users SET role = ...
DELETE FROM	Supprimer une ligne	DELETE FROM Users WHERE ...
COUNT, SUM, AVG...	Fonctions d'agrégation	SELECT COUNT(*) FROM Users

Erreurs fréquentes

- Oublier la condition de jointure → crée un produit cartésien
- Confondre = NULL (incorrect) et IS NULL (correct)
- Oublier WHERE dans un UPDATE ou DELETE → modifie TOUTE la table
- Respecter l'ordre : SELECT ... FROM ... WHERE ... ORDER BY ...
- Créer les tables dans le bon ordre (les tables sans clé étrangère d'abord)

• • •

*Cours SQL — **Traineau Julien***
Bases de données relationnelles · Terminale